

Diseño y Análisis de Algoritmos

Set de Problemas 5

Profesor: Felipe Ruiz

Ayudante: Cristian Nettle

1. Distancia mínima en una red social

Considere una red social modelada como un grafo simple, no dirigido y no ponderado $G = (V, E)$, donde cada vértice representa a una persona y cada arista $(u, v) \in E$ indica que las personas u y v son amigas.

Dadas dos personas $s, t \in V$, se desea calcular la distancia mínima en número de “saltos de amistad” entre s y t , es decir, la longitud del camino más corto (en número de aristas) que conecta s con t .

Diseñe un algoritmo basado en *búsqueda en anchura* (BFS) que, dados G , s y t , compute la distancia mínima entre s y t , o determine que no existe un camino entre ellas.

2. Contar islas en una matriz binaria

Se tiene una matriz binaria M de tamaño $n \times m$, donde cada celda $M[i][j]$ contiene un 0 o un 1. Una *isla* se define como un conjunto máximo (por inclusión) de celdas con valor 1 tal que cualquier par de ellas está conectado mediante una secuencia de celdas adyacentes con valor 1. Considere adyacencia de 4 vecinos (arriba, abajo, izquierda, derecha).

1. Defina formalmente qué significa que dos celdas sean adyacentes y pertenezcan a la misma isla, bajo la noción de adyacencia por 4 vecinos.
2. Diseñe un algoritmo basado en BFS o DFS que reciba M y retorne el número total de islas presentes en la matriz.
3. Escriba el pseudocódigo de su solución e indique explícitamente cómo maneja la estructura de *visitados*.

3. Detección de ciclos dirigidos

Sea $G = (V, E)$ un grafo dirigido. Se desea determinar si G contiene algún ciclo dirigido, es decir, una secuencia de vértices distintos (excepto el primero y el último)

$$v_0, v_1, \dots, v_k$$

con $k \geq 1$ tal que $(v_i, v_{i+1}) \in E$ para todo $0 \leq i < k$ y $v_0 = v_k$.

Diseñe un algoritmo basado en *búsqueda en profundidad* (DFS) que determine si existe al menos un ciclo dirigido en G .

Puede utilizar el esquema clásico de colores (blanco, gris, negro) o, alternativamente, apoyarse en la pila de recursión para distinguir entre vértices no visitados, en proceso y finalizados.

Entregue el pseudocódigo de su algoritmo, indicando claramente:

- Cómo se marcan o clasifican los vértices (colores o estados).
- En qué condición se concluye que existe un ciclo.

4. Conectividad en grafos no dirigidos

Dado un grafo no dirigido $G = (V, E)$, se desea determinar si es conexo, es decir, si para cualquier par de vértices $u, v \in V$ existe un camino que conecte u con v .

Diseñe un algoritmo basado en BFS o DFS que determine si G es conexo.

4. Elemento mayoritario en un arreglo

Sea un arreglo A de n elementos (no necesariamente ordenados). Se dice que un elemento x es *mayoritario* si aparece en A más de $n/2$ veces.

Diseñe un algoritmo basado en *hashing* que determine si existe un elemento mayoritario en A y, en caso afirmativo, lo retorne.

Su algoritmo debe ejecutarse en tiempo esperado $O(n)$. Entregue el pseudocódigo de su solución y analice su complejidad temporal y espacial.